Week 10 · Verify, hand over, present

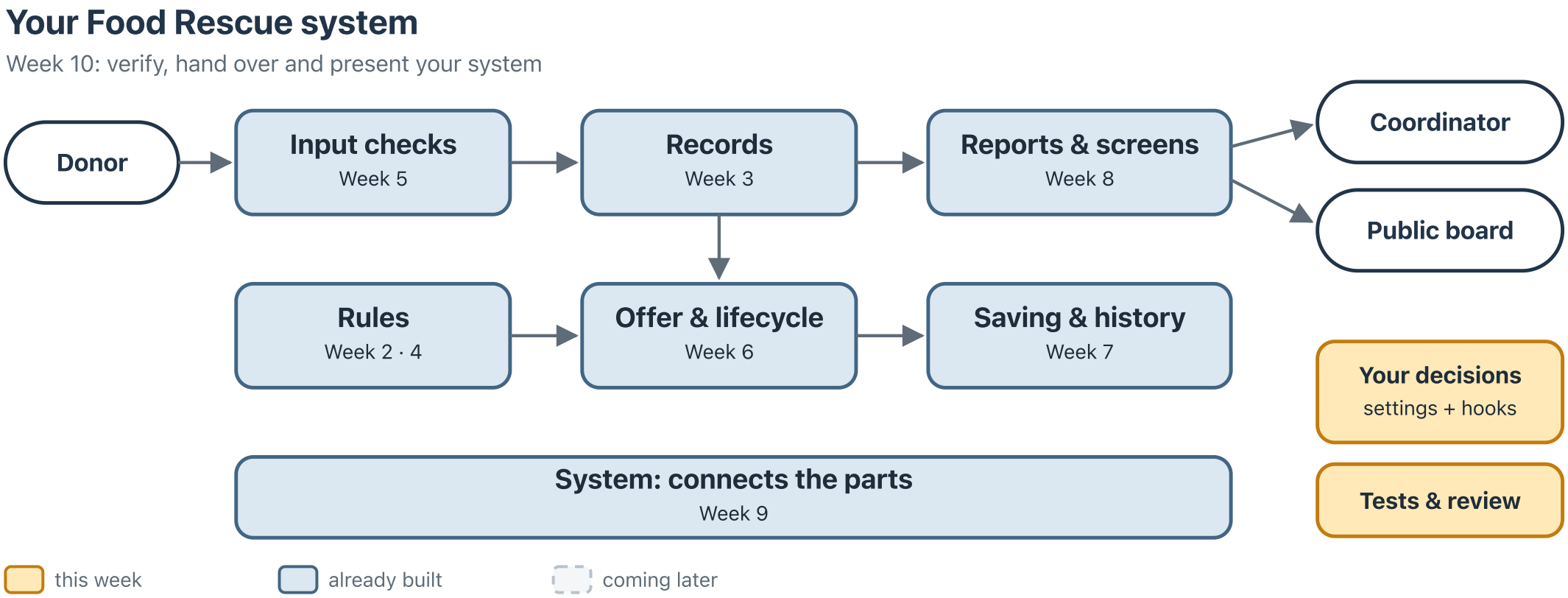
Programming for Information Systems

How do you know it works, and could someone run it without you?

This week

Question	How do you know your system works, and could someone run it without you?
Python	test functions, reading test output, clean runs, reviewing code
Principle 10	Verify before you trust
You build	tests for your own decisions, a handoff note, your finished Final

Your system, by Week 10



The idea: a system someone else can run

- A shop owner goes away. A friend runs the shop from a short **note**.
- The friend tries it on the first morning, while the owner is still reachable.
- In information systems this is a **handoff**.

> Your system is **verified** when: **1** the tests pass · **2** a clean run works · **3** a peer can run it from your instructions alone.

Verification

Tests, output, clean runs, review

A test checks one promise

```
def discount(price):
    if price >= 100:
        return 10
    return 0

def test_discount_at_the_boundary():
    assert discount(100) == 10, "exactly 100 gets the discount"
```

- The name starts with `test_` and **says the promise**.
- Pass → nothing printed. Fail → `AssertionError` with your message.

Run many tests without stopping

```
def run_tests(tests):
    passed = 0
    for test in tests:
        try:
            test()
            print("✅", test.__name__)
            passed = passed + 1
        except Exception as error:
            print("❌", test.__name__, "→", error)
    print(passed, "of", len(tests), "tests passed")
```

Reading test output

```
✅ test_public_board_hides_contacts
❌ test_25_portions_need_a_team → AssertionError:
   25 portions is my team minimum, so it needs a team
2 of 3 tests passed
```

1. The **last line**: how many failed?
2. The **name**: which promise is broken?
3. The **message**: what should be true?
4. **Decide:** is the code wrong, the test wrong, or did a decision change?

A clean run: Runtime → Restart session and run all

- A notebook remembers cells you **changed or deleted**.
- It can look fine for you and fail for everyone else.
- **Colab:** Runtime → Restart session and run all.
- Every cell runs top to bottom, with no error. Then read the outputs again.

```
total_today = 48     # typed in a cell that was later deleted
# a fresh process: NameError: name 'total_today' is not defined
```

The 10 principles as a review checklist

#	Principle	Ask
1	Clear names and units	Does every name say what it means, and in what unit?
2	Explicit rules	Named numbers? Every case? Boundary tested?
3	One record per real thing	Is each fact stored once, with one ID?
4	One function, one job	Does it return instead of print?
5	Never trust input	Is bad input refused, explained, and nothing changed?
6	Wrong states hard to reach	Can it skip approval, or go backwards?
7	Never lose data	Could anything be overwritten or forgotten?
8	Right information, right person	Private data safe? Numbers honest?
9	Separate what changes	Is a decision buried in the machinery?
10	Verify before you trust	Is there a test? Did it run from a clean start?

Reviewing AI-written code

1. **Read the request.** What should the code promise?
2. **Read the code** with the checklist. Note the problem and the line.
3. **Test the edges:** normal, boundary, odd (empty, zero, missing).
4. **Plug it in and watch for ⚠️.** A fallback means your code is *not* used.
5. **Decide:** accept, fix or reject, and write down why.

> "It runs" is not the same as "it is correct".

⚖️ Good or risky?

Version A (from an AI assistant)

```
def ai_pickup_label(portions, approved):
    """Improved pickup label."""
    if portions >= 25:
        print("team")
    else:
        print("standard")
```

Version B (your Week 2 rule)

```
def pickup_label(portions, approved):
    if not approved:
        return "hold"
    if portions >= TEAM_PICKUP_MIN_PORTIONS:
        return "team"
    return "standard"
```

"*I tested it and it works.*"

What the review finds

- **It returns None** → the system falls back to the default. The report *looks* right, but the AI's code is **never used**.
- **Ignores approval** → a new offer would be `team`, not `hold`. Volunteers drive out for food that may be rejected.
- **Magic number 25** → the coordinator's setting is ignored; screens disagree.
- **No test** → `fr.check` shows ❌ at once.

> Want a team from 25? Change **one setting**. No new code.

You build: tests that protect your decisions

```
def test_offer_without_contact_is_refused():
    system = fresh_system()
    row = offer_row(system, 10)
    row["contact"] = "  "
    offer_id, errors = system.add_offer(row)
    assert offer_id is None, "contact is required in my system"
```

- Two tests for **one** decision: the boundary, and the other side.
- Read numbers from `system.settings`, not from memory.

Run the full verification

```
final_check = fr.System(MY_SETTINGS, hooks=MY_HOOKS)
final_check.verify()
run_tests(example_tests + my_tests)
```

- **Kit checks:** must all be ✅.
- **Your system:** your decisions and hooks, and any ⚠️ to explain.
- **Your tests:** your promises, all ✅.

The handoff note

What my system does	who uses it, for what (2 sentences)
How to run it	numbered steps, and what you should see
My decisions	the 3 most important: setting, value, why
What I tested, and how	tests, clean run
Known limits	what it does not do; any ⚠️

Session 2 · The operator demonstration

The author	The operator
Gives the notebook and the handoff note	Opens it on **their own** computer
Watches **silently**; no pointing	Follows **only** the note, thinking aloud
Notes where the operator hesitates	Fills in the peer test record

- Then fix the note, retest that step, and swap roles.

Peer test record

Step in the note	What the operator did	What happened	Note
1		worked / needed help / did not work	
2			
3			

- Also: the first difficulty · did the author speak? · `verify()` result · one change made · retested?

A 3-minute presentation

Part	Time
1 · The people and the problem	0:30
2 · My decisions (value, alternative, who is affected)	0:45
3 · A demo: one output your decisions changed	0:45
4 · What I tested	0:30
5 · What I would improve next	0:30

Session 3 · Individual verification

- You explain **one decision** and **one test** to the instructor.
- Be ready to run your test, and to say what change would make it fail.
- The instructor may give you an **unseen offer**: predict first, then run.
- Everyone else finishes and submits the Final.

Submit the Final project

- `Final_Project.ipynb`: **Runtime → Restart session and run all**, no errors.
- `verify()` all ✅ · at least 3 decisions with a decision log.
- Section 5: your two decision tests ✅, handoff note, peer test record.
- Email **submit.progmis@gmail.com**. Deadline and file name are announced in class.

What you can do now

- Read code, test it and **judge** it, including code an AI writes for you.
- Keep asking two questions:

> *Who is affected?* and *How do I know it works?*